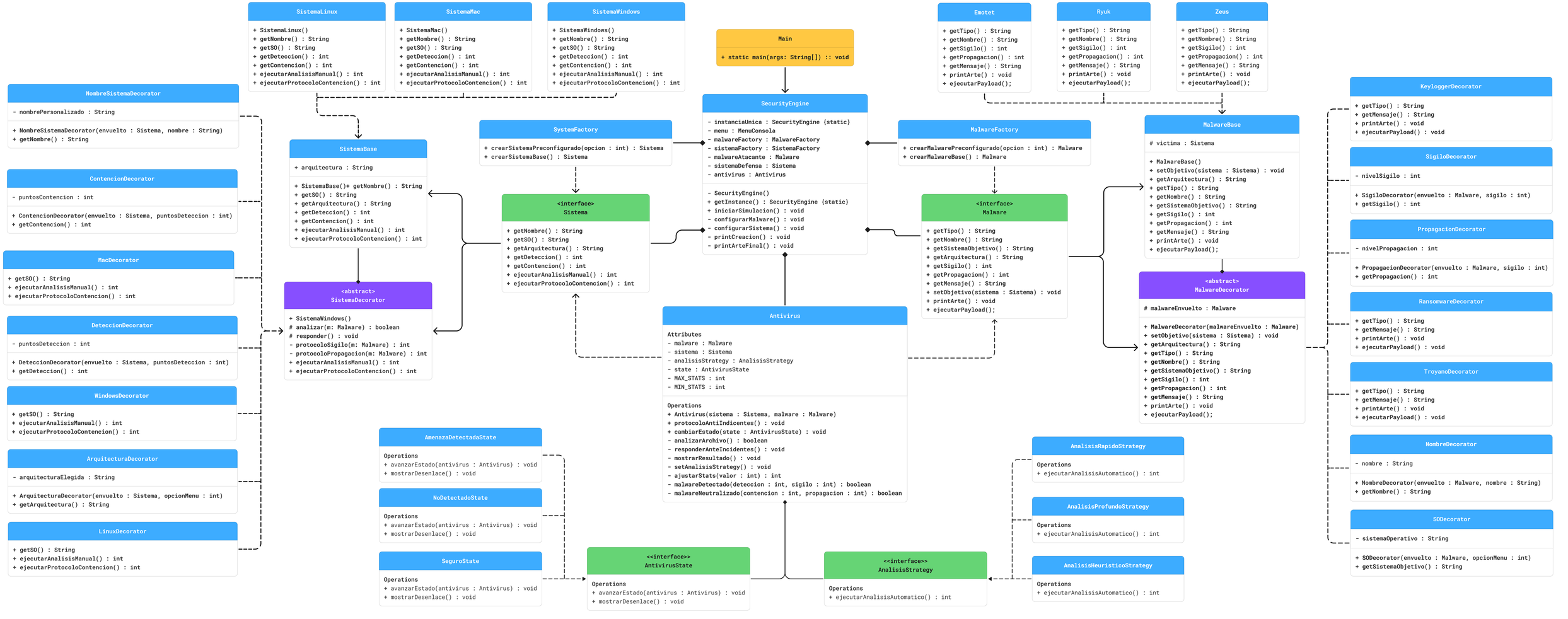
SistemaLinux
+ SistemaLinux()
+ getNombre() : String
+ getSO() : String
+ getDeteccion() : int
+ getContencion() : int
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
SistemaMac
+ SistemaMac()
+ getNombre() : String
+ getSO() : String
+ getDeteccion() : int
+ getContencion() : int
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
SistemaWindows
+ SistemaWindows()
+ getNombre() : String
+ getSO() : String
+ getDeteccion() : int
+ getContencion() : int
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
Main
+ static main(args: String[]) :: void
Emotet
+ getTipo() : String
+ getNombre() : String
+ getSigilo() : int
+ getPropagacion() : int
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload();
Ryuk
+ getTipo() : String
+ getNombre() : String
+ getSigilo() : int
+ getPropagacion() : int
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload();
Zeus
+ getTipo() : String
+ getNombre() : String
+ getSigilo() : int
+ getPropagacion() : int
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload();
NombreSistemaDecorator
- nombrePersonalizado : String
+ NombreSistemaDecorator(envuelto : Sistema, nombre : String)
+ getNombre() : String
ContencionDecorator
- puntosContencion : int
+ ContencionDecorator(envuelto : Sistema, puntosDeteccion : int)
+ getContencion() : int
MacDecorator
+ getSO() : String
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
DeteccionDecorator
- puntosDeteccion : int
+ DeteccionDecorator(envuelto : Sistema, puntosDeteccion : int)
+ getDeteccion() : int
WindowsDecorator
+ getSO() : String
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
ArquitecturaDecorator
- arquitecturaElegida : String
+ ArquitecturaDecorator(envuelto : Sistema, opcionMenu : int)
+ getArquitectura() : String
LinuxDecorator
+ getSO() : String
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
SistemaBase
+ arquitectura : String
+ SistemaBase()+ getNombre() : String
+ getSO() : String
+ getArquitectura() : String
+ getDeteccion() : int
+ getContencion() : int
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
<abstract>
SistemaDecorator
+ SistemaWindows()
# analizar(m: Malware) : boolean
# responder() : void
- protocoloSigilo(m: Malware) : int
- protocoloPropagacion(m: Malware) : int
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
SystemFactory
+ crearSistemaPreconfigurado(opcion : int) : Sistema
+ crearSistemaBase() : Sistema
<interface>
Sistema
+ getNombre() : String
+ getSO() : String
+ getArquitectura() : String
+ getDeteccion() : int
+ getContencion() : int
+ ejecutarAnalisisManual() : int
+ ejecutarProtocoloContencion() : int
SecurityEngine
- instanciaUnica : SecurityEngine {static}
- menu : MenuConsola
- malwareFactory : MalwareFactory
- sistemaFactory : SistemaFactory
- malwareAtacante : Malware
- sistemaDefensa : Sistema
- antivirus : Antivirus
- SecurityEngine()
+ getInstance() : SecurityEngine {static}
+ iniciarSimulacion() : void
- configurarMalware() : void
- configurarSistema() : void
- printCreacion() : void
- printArteFinal() : void
MalwareFactory
+ crearMalwarePreconfigurado(opcion : int) : Malware
+ crearMalwareBase() : Malware
<interface>
Malware
+ getTipo() : String
+ getNombre() : String
+ getSistemaObjetivo() : String
+ getArquitectura() : String
+ getSigilo() : int
+ getPropagacion() : int
+ getMensaje() : String
+ setObjetivo(sistema : Sistema) : void
+ printArte() : void
+ ejecutarPayload();
MalwareBase
victima : Sistema
+ MalwareBase()
+ setObjetivo(sistema : Sistema) : void
+ getArquitectura() : String
+ getTipo() : String
+ getNombre() : String
+ getSistemaObjetivo() : String
+ getSigilo() : int
+ getPropagacion() : int
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload();
<abstract>
MalwareDecorator
malwareEnvuelto : Malware
+ MalwareDecorator(malwareEnvuelto : Malware)
+ setObjetivo(sistema : Sistema) : void
+ getArquitectura() : String
+ getTipo() : String
+ getNombre() : String
+ getSistemaObjetivo() : String
+ getSigilo() : int
+ getPropagacion() : int
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload();
KeyloggerDecorator
+ getTipo() : String
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload() : void
SigiloDecorator
- nivelSigilo : int
+ SigiloDecorator(envuelto : Malware, sigilo : int)
+ getSigilo() : int
PropagacionDecorator
- nivelPropagacion : int
+ PropagacionDecorator(envuelto : Malware, sigilo : int)
+ getPropagacion() : int
RansomwareDecorator
+ getTipo() : String
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload() : void
TroyanoDecorator
+ getTipo() : String
+ getMensaje() : String
+ printArte() : void
+ ejecutarPayload() : void
NombreDecorator
- nombre : String
+ NombreDecorator(envuelto : Malware, nombre : String)
+ getNombre() : String
SODecorator
- sistemaOperativo : String
+ SODecorator(envuelto : Malware, opcionMenu : int)
+ getSistemaObjetivo() : String
Antivirus
Attributes
- malware : Malware
- sistema : Sistema
- analisisStrategy : AnalisisStrategy
- state : AntivirusState
- MAX_STATS : int
- MIN_STATS : int
Operations
+ Antivirus(sistema : Sistema, malware : Malware)
+ protocoloAntiIndicentes() : void
+ cambiarEstado(state : AntivirusState) : void
- analizarArchivo() : boolean
- responderAnteIncidentes() : void
- mostrarResultado() : void
- setAnalisisStrategy() : void
- ajustarStats(valor : int) : int
- malwareDetectado(deteccion : int, sigilo : int) : boolean
- malwareNeutralizado(contencion : int, propagacion : int) : boolean
AmenazaDetectadaState
Operations
+ avanzarEstado(antivirus : Antivirus) : void
+ mostrarDesenlace() : void
NoDetectadoState
Operations
+ avanzarEstado(antivirus : Antivirus) : void
+ mostrarDesenlace() : void
SeguroState
Operations
+ avanzarEstado(antivirus : Antivirus) : void
+ mostrarDesenlace() : void
<<interface>>
AntivirusState
Operations
+ avanzarEstado(antivirus : Antivirus) : void
+ mostrarDesenlace() : void
<<interface>>
AnalisisStrategy
Operations
+ ejecutarAnalisisAutomatico() : int
AnalisisRapidoStrategy
Operations
+ ejecutarAnalisisAutomatico() : int
AnalisisProfundoStrategy
Operations
+ ejecutarAnalisisAutomatico() : int
AnalisisHeuristicoStrategy
Operations
+ ejecutarAnalisisAutomatico() : int